

Software demo: run sheet

MSc defence, Friday 28 August 2026, 10:00-11:15, DTU Lyngby building 322 room 017. Joint presentation of max. 30 minutes, of which approx. 5 minutes is a live demonstration of the software. Christian Wu and Daniel Diamant, supervisor Professor Carsten Witt.

1. The two commands

This is the entire demo. Everything else in this document is preparation or narration.

```
cd ~/demo
uv run run.py cloud -a SA GA UMDA greedy -s 1
uv run run.py ev -a SA Greedy -s 1
```

run.py handles the difference between the two modules for you: the cloud module runs from its own directory, the EV module runs from the project root with PYTHONPATH set. That is the only reason the launcher exists, and it is worth one sentence on stage.

Case matters and differs between the modules: cloud takes greedy in lower case, EV takes Greedy capitalised.

2. Setup, the evening before, on the presentation machine

```
# work from a copy, so a demo run does not leave your results dirty
cp -R "/path/to/Master_Thesis_Metaheuristics" ~/demo
cd ~/demo

# run both twice: the first pass downloads packages and warms the
# uv cache, the second pass is the real timing to plan around
time uv run run.py cloud -a SA GA UMDA greedy -s 1
time uv run run.py ev -a SA Greedy -s 1
```

Write down what time reports on the second pass. Those two numbers are what the run sheet in section 5 should be built around. Measured on a Windows laptop the runs take 79 s and 93 s warm, against 100 s cold; on the Apple M3 Pro that produced the committed results, expect roughly 15 s and 25 s. Measure, do not assume.

The copy is hygiene, not a safety rule. A run overwrites the results directory it writes to, so running in the real tree leaves it dirty and, if committed, puts 1-seed numbers in a repository whose README says its results came from full runs. Nothing you project comes from those files, and git checkout undoes it in two seconds. One cd avoids the question entirely.

3. Machine and logistics

Use **one laptop for the whole demo**, the Mac that produced the committed results. Swapping laptops mid-demo costs 30 to 60 seconds of dead air out of 300 and is the most common way a two-person demo falls apart.

Item	Do this	Why
Power	Plug in, disable Low Power Mode	macOS throttles hard on battery; this alone can turn 15 s into 60 s
Background apps	Quit Docker, Chrome, Slack	The runs are single-threaded but memory and thermal pressure still bite
Windows layout	Terminal left half, image viewer right half, set before you walk in	Never resize or rearrange anything live
GIF playback	Open the GIFs in a browser tab, not Preview.app	Preview treats a GIF as a multi-page document and shows a still frame. Quick Look (spacebar) animates; verify on the actual machine
Terminal font	16 to 18 pt	It is a lecture room, not your desk
Fallback	Screen-record the full 5 minutes the day before	If the projector, laptop or network fails you play the recording and lose nothing

Item	Do this	Why
Recovery tab	A second terminal tab holding a completed run, scrolled to the summary	Instant recovery if anything hangs on stage
Windows only	<code>export PYTHONIOENCODING=utf-8</code>	EV main.py crashes with UnicodeEncodeError on the Wilcoxon header in cp1252. Not an issue on macOS

4. Files to have open before you start

```
~/demo/Cloud_scheduling/figures/balanced/allocation_greedy_construction.gif
~/demo/Cloud_scheduling/figures/balanced/allocation_sa_search.gif
~/demo/Cloud_scheduling/results/balanced/run_manifest.yaml
~/demo/report/graphics/sf75_instance_map.png
~/demo/EV_routing/results/sf_75/figures/route_comparison_map.png
~/demo/EV_routing/results/sf_75/summary.md
```

5. The five minutes, minute by minute

Timings assume the Mac. On the Mac each run finishes before you finish the sentence that launched it, so the structure is **result first, explanation second**. Do not try to talk over the run to fill time; there is no time to fill.

Christian takes cloud scheduling, Daniel takes EV routing, which also satisfies the equal-speaking-time requirement.

Time	Who	What happens	What you say
0:00	C	Paste the cloud command. Lands in about 15 s.	"Fifty tasks, ten servers, one seed, a hundred and fifty thousand objective evaluations."
0:30	C	Point at SA best = 1.8086, thesis table beside it.	"That is the same number to four decimals as the table in chapter 6. Seed 0 happens to be the best of our twenty seeds."
0:45	C	Greedy construction GIF (27 s), then SA search GIF (17 s).	"Each rack is a server, height is CPU capacity, colour is task priority. This is what those evaluations were actually doing."
1:30	C	One scroll of run_manifest.yaml.	"Every run writes a manifest next to its results: CLI arguments, calibrated constants, every hyperparameter. That is how any number in the thesis traces back to the run that produced it."
1:50	-	Handover.	Agree the exact sentence in advance.
2:10	D	Paste the EV command. Lands in about 25 s.	"Seventy-five customers and thirty real charging stations in San Francisco, road distances from OSRM, elevations from SRTM."
2:40	D	Instance map, then route comparison map.	"Greedy goes 309 km and burns 111 kWh. SA goes 258 km on 78 kWh and pays \$9.73 in charging instead of \$24.10. Thirty percent less energy, and the charging bill drops by sixty percent."
3:50	D	summary.md, scrolled to the ranking.	"Same protocol as the cloud problem: fixed evaluation budget, frozen tuned parameters, twenty seeds in the thesis, Wilcoxon with Holm correction."
4:20	C/D	Results directory layout, 33 unit tests, README.	"Both modules follow the same protocol and both are reproducible from the repository."
4:40	-	Buffer.	Stop talking. Do not fill it.

6. Numbers you are allowed to quote

The live run uses **one seed**. The thesis reports **twenty**. For cloud these coincide; for EV they do not. Say "one seed" out loud on the EV half, otherwise a censor with the table open will catch the mismatch.

Quantity	Live 1-seed run	Thesis value	How to phrase it
Cloud SA	1.8086	1.8086 (best of 20)	"Identical. Seed 0 is our best seed."
Cloud greedy BFD	2.0062	2.0062	"Deterministic, so it is exact."
Cloud improvement	+9.9 %	+9.9 %	"Search beats construction by ten percent."

Quantity	Live 1-seed run	Thesis value	How to phrase it
EV greedy	3.5969	3.5969	"Deterministic, exact match."
EV SA	2.5044	2.4649 best, 2.5583 mean	"One seed, so it sits between our twenty-seed best and mean."
EV improvement	+30.4 %	+31.0 %	"About thirty percent over the baseline."

7. What actually happens when you press enter

Phase by phase, so you can narrate confidently and answer "what is it doing right now". Times are the Windows measurements; divide by roughly five for the Mac.

Cloud scheduling, approx. 79 s Windows / 15 s Mac

Phase	On screen	Say this, and why it matters
Instance	50 tasks, 10 servers, 2256.7 % CPU demand against 4300 % capacity.	The instance is oversubscribed enough that placement matters, but it is feasible. This frames the whole problem in one line.
Calibration (2 s)	25/150 feasible samples. $E_{ref} = 12572$ W, $L_{ref} = 29267$ ms, $\lambda = 206.47$.	The best technical moment of the cloud half. Watts and milliseconds are not comparable, so the objective is normalised by references sampled from the instance itself, and the capacity penalty is set to 100x the worst feasible objective so no infeasible solution can ever beat a feasible one.
Focus mode	BALANCED, $w_{energy} = w_{latency} = 1.0$.	One switch in config.yaml moves this to eco or performance. The objective is configuration, not code.
SA diagnostic (16 s)	$F(X) = 1.8146$, acceptance 30.55 %, 6 reheats, per-server task histogram (19, 8, 6, 5, 3, 2, 2, 2, 2, 1).	The histogram is the most legible thing in the whole log. That skew is the consolidation-versus-congestion trade-off made visible. Note SA runs twice: this diagnostic, then the seeded comparison.
Multi-seed (50 s)	One line per algorithm: SA, GA, UMDA, greedy.	All three metaheuristics under an identical 150,000-evaluation budget. Equal budget is what makes the comparison fair.
Summary	SA 1.81, GA 1.82, UMDA 1.82, greedy 2.01. Energy contributes 55 %, latency 45 %.	SA wins, but the margin over GA and UMDA is under one percent, and with one seed you cannot claim significance. Say so before you are asked.
Outputs (10 s)	4 figures, 3 CSVs, run_manifest.yaml, summary.md.	Every run is self-documenting. This is the reproducibility argument.

EV routing, approx. 93 s Windows / 25 s Mac

Phase	On screen	Say this, and why it matters
Load	Weights loaded from weights.json, multipliers all 1.0.	Weights were calibrated in a separate, committed step and then frozen. The run does not tune itself, which is what makes it a controlled comparison.
SA diagnostic (35 s)	Objective 2.5815, 273 km, 82.2 kWh, acceptance 14.9 %, 7 reheats.	Acceptance is 14.9 % here against 30.6 % in the cloud problem. The EV landscape is far more constrained, because battery feasibility rejects most moves.
Greedy (0.00 s)	3.5969, 309.2 km, 111.3 kWh, 1.15 h charging, \$24.10.	Nearest-neighbour with a charging repair. Instant, feasible, and clearly poor. This is the number everything else is measured against.
Comparison (42 s)	SA seed 0 lands on 2.5044.	Same 150,000-evaluation budget as the cloud problem. One protocol, two problems.
Interpretation	+30.4 % over greedy. SA best run: 258.5 km, 77.7 kWh, \$9.73.	Quote the physical numbers, not the objective value. "Thirty percent less energy and a sixty percent smaller charging bill" lands with a censor. "2.50 versus 3.60" does not.
Outputs (10 s)	5 figures, 3 CSVs, manifest, summary.md.	Identical output contract to the cloud module, written independently.

8. Five things on screen that can bite you

All five are real, all five were observed in a rehearsal run. None is a bug in the science, but each one looks bad on a projector if it surprises you.

1. The Wilcoxon table at one seed says "no sig. difference", $p = 1.0000$.

With $n = 1$ the signed-rank test has no power at all, so $W = 0$ and $p = 1$. Projected on a wall, "no significant difference between Greedy and SA" reads as your method failing. **Say the sentence before it appears:** "one seed, so the test is a placeholder here; the thesis runs twenty and the difference is significant after Holm correction." Alternatively run the EV demo with `-s 3`, which costs about 30 extra seconds on the Mac and produces a table that does not say that.

2. The printed LaTeX table is captioned "(10 seeds, mean +/- std)" whatever you ran.

The caption default is hard-coded in `EV_routing/tools/statistics.py` line 196 and does not follow the actual seed count, so on a 1-seed run it contradicts the "1 seeds" header two lines above it. Do not leave that block on screen. If asked, it is a cosmetic default in a convenience helper that writes a table for pasting into the report, and the reported tables were produced from 20-seed runs.

3. Two different SA numbers appear in the EV run: 2.5815, then 2.5044.

The first is the diagnostic call at `EV_routing/main.py` line 1288, which is not given a seed. The second is seed 0 of the controlled comparison. Know this before somebody asks why SA produced two answers. The seeded one is the number that goes in the results.

4. The EV run ends by dumping both full routes.

Eighty-five node IDs each, filling the screen, and it is the last thing printed. The summary table you want to point at is scrolled well above it. Either scroll back up before you start talking, or plan to finish on `summary.md` instead.

5. A matplotlib UserWarning prints near the top of the EV run.

`set_ticklabels()` from `tools/plot.py` line 350. Harmless, but it is yellow and it is the first thing on the screen. Ignore it; if asked, it is a tick-locator warning from a figure helper.

9. Why the demo is built this way

Choice	Reason
One seed, not twenty	Twenty seeds is 3.5 minutes for cloud and 39 minutes for EV. One seed is the same code, the same budget and the same protocol, finishing inside the time available. The full runs are already committed, so the demo shows the machinery and the repository shows the evidence.
Real entry point, not a demo script	The credibility of a software demo is the examiner seeing the documented command invoked. A wrapper would hide exactly the thing worth witnessing, and would be new unrehearsed code introduced days before the defence into the one part of the event with live failure risk.
SA, GA, UMDA for cloud but only SA for EV	Cloud has three metaheuristics at 15 s each on the Mac, so all three fit. In EV, GA and ACO cost 43 s and 54 s per seed even on the M3 and would take the whole five minutes on their own. SA is the winner in both problems, so it is the one to show.
Animations pre-rendered, not generated live	Rendering the SA GIF takes about a minute even at reduced budget. It is generated by <code>visualize.py --animate sa</code> from the same code path as the results, so say the command out loud rather than pretend it is live.
Sandbox copy	A run overwrites the results directory it writes to. The committed results back the tables in the thesis. Those two facts are why the demo runs in <code>~/demo</code> .
Physical units in the narration	Kilometres, kilowatt-hours and dollars are immediately meaningful. A normalised objective value is not, and inviting a question about how $F(X)$ is scaled in the middle of a five-minute demo is a poor trade.

Generated by `make_demo_guide.py`. Runtimes measured 19 August 2026: Windows 11 laptop for the absolute figures, per-algorithm Apple M3 Pro times taken from the committed `results_summary.csv`. Re-measure on the presentation machine.